

Technical Report: Network Threat Detection Through Anomalies and Attack Classification

Farah Ahmed

Data Science MS Online, Boston University

DX 799 O2: Data Science Capstone

Joshua Von Korff

April 19, 2026

Executive Summary

As digital infrastructure continues to expand, cybersecurity threats are increasing in number and sophistication, making them more difficult to detect and mitigate. As a result, organizations face pressure to prioritize security measures that effectively monitor network traffic to detect malicious activity and prevent cyberattacks. However, traditional security and detection systems are insufficient for evolving cyber threats, making the need for adaptive machine learning strategies capable of detecting both known and emerging threats. This project's goal is to develop models that can detect and classify suspicious or attack behavior and address the guiding research question: "Which types of cyberattacks are the most common?" To accomplish this, the BPF-extended Tracking Honeypot (BETH), which captures real-world DNS traffic for anomaly detection and classification, and the UNSW-NB15 Network Intrusion Detection dataset, which provides labeled network traffic for supervised classification of various attack types, were used. The two datasets provided complementary perspectives on network activity. Techniques such as preprocessing, feature selection, and normalization were applied to provide robust model performance and stability. Supervised modeling techniques, such as Logistic Regression, Random Forest, and Gradient Boosting, were implemented for classification tasks. Particularly, ensemble tree-based models were developed to better capture non-linear patterns in the datasets. Also, unsupervised methods, such as K-Means and DBSCAN, were used to uncover anomalous patterns. A key finding of this study is that ensemble tree-based models consistently outperform linear models. Random Forest achieved strong performance on the BETH dataset, with a recall of 0.950 and an F1 score of 0.970, indicating high effectiveness in detecting suspicious activity. On UNSW-NB15, HistGradientBoosting achieved the best performance, with a recall of 0.920 and an F1 score of 0.924, demonstrating a strong ability to

identify attack behavior. Logistic regression showed more balanced performance and generalized better to unseen data, but it reflected limitations in capturing complex patterns and feature relationships effectively. These results indicate that ensemble tree-based models are highly effective for intrusion detection tasks, specifically dealing with imbalanced and high-dimensional data. Based on this analysis, recommendations include using data-driven insights to strengthen real-time monitoring and threat detection systems, adopting generalizable predictive models to prevent cyberattacks, and continuously maintaining and improving detection measures to address emerging threats. Specifically, organizations should consider implementing ensemble tree-based models into their security monitoring pipelines, as these models can effectively detect suspicious or attack behavior and complex patterns. This can help businesses enhance their security systems and make informed, proactive decisions about threat mitigation. Stakeholders can use these insights to strengthen their defense and response strategies and optimize resource deployment, ensuring networks are resilient to security threats.

Introduction and Problem Definitions

Background and Domain Context

Cybersecurity is a critical domain as it aims to protect networks, systems, and data from attacks that compromise sensitive information and disrupt vital system operations. Cyber threats are rapidly growing and becoming more complex as digital infrastructure expands and reliance on interconnected systems increases. This expansion has increased system exposure, creating opportunities for threat actors to exploit vulnerabilities across networks, applications, and user-level environments. Suspicious or malicious acts are used to disrupt and exploit system vulnerabilities, potentially damaging the integrity of data, interrupting critical operations, and violating privacy. Common techniques used by threat actors include phishing, malware,

distributed denial-of-service (DDoS), and network or system intrusions. These attacks are highly sophisticated, often involving a multi-step process to break into a system, the use of encryption to hide communication, and employing tactics that avoid detection. Standard security measures use legacy tools that incorporate manual threat detection and reliance on known threat signatures, which often leave systems vulnerable to emerging and complex threats. Also, they struggle to adapt well to environments where network behavior is continuously changing. Hence, organizations face challenges in detecting attacks that exploit unknown flaws in security systems and anomalies that do not match typical threat signatures. Machine learning is a powerful, robust tool that enhances cybersecurity by automatically identifying abnormal behavior and patterns, adapting to evolving threats, and detecting those that traditional methods might otherwise miss.

Client Goals/Objectives and Research Question

The primary objective of this project is to build and assess machine learning models on their ability to detect and classify malicious activity accurately, using datasets that capture real-world DNS traffic, system processes, and cyberattack patterns. This project aims to improve intrusion detection performance while also providing insights into the distribution and characteristics of cyberattacks. The following objectives and research questions are designed to provide actionable insights for organizations in their decision-making process.

From a client perspective, the intended outcome is a data-driven approach for intrusion detection that is supported by:

- Real-time monitoring of network and system activity
- Early detection of suspicious, malicious, and attack behaviors
- Classification of different types of cyberattacks
- Improved prioritization of responses and strategies based on threat type

To address this, the central research question guiding the project's analysis is:

- “Which types of cyberattacks are the most common?”

This question is created to determine the frequency distributions of different cyberattacks and to help improve the allocation of cybersecurity resources and threat prioritization.

Additional Research Questions/Objectives

To further support the primary research question and the client's goals, additional questions were explored, including:

1. How does preprocessing, including feature selection and handling class imbalance, impact model performance?
2. Which machine learning models demonstrate the strongest performance in detecting malicious or suspicious network behavior?
3. Which features are most indicative of malicious or suspicious activity?

These questions were designed to provide a comprehensive evaluation of model performance and to identify which model architectures and preprocessing strategies yield the most accurate and reliable results.

Success Criteria

The success of the analysis is evaluated by using a variety of quantitative performance metrics and model interpretability outcomes. Metrics such as accuracy, precision, ROC-AUC, F1 score, and macro F1 score were used. Due to the presence of severe class imbalance in both datasets, recall and macro F1 are prioritized over accuracy. Recall is emphasized because it reduces the number of false negatives, which, if not addressed, could lead to data breaches. Macro F1 is used to ensure a more balanced performance across the majority and minority classes, thereby providing a more reliable measure of model performance in imbalanced datasets.

Model success is also determined by their ability to identify features strongly associated with malicious or suspicious behavior, generalize to unseen data, and distinguish between different attack types, all of which provide insights into patterns and help mitigate cyber threats.

Data Overview

Data Sources and Collection Methods

This project uses two datasets from the *Cybersecurity Project: Detecting Network Threats Through Anomaly Detection and Attack Classification*. The BPF-extended Tracking Honeypot (BETH) dataset contains over 8 million records of cyberattacks and host activities gathered from over 23 honeypots. It captures kernel-level process logs and real-world DNS traffic with binary target labels of suspicious and malicious behavior. This dataset is best suited for anomaly detection and classification. The UNSW-NB15 dataset contains both real and synthetic network traffic data with binary and multiclass labels. It includes 175,341 network events and 49 features, which represent various network traffic characteristics. This dataset is widely used as a benchmark for evaluating network intrusion detection under both binary and multi-class classification tasks. An additional dataset, the Cybersecurity Attacks dataset, was initially evaluated for analysis, but was later excluded due to its synthetic data, poor model performance, and weak relationship between its input features and the target variable. The datasets were collected from Kaggle and downloaded in CSV format. Preprocessing, exploratory data analysis (EDA), and modeling were performed using Python in Visual Studio Code.

Data Description and Dictionary

The BETH dataset is composed of kernel-level process calls and network traffic. Each record consists of 14 features and 2 labels, with features of a mix of categorical and numerical types. It contains a subset of 1,141,078 records, split into training, testing, and validation sets.

The training set comprises 763,144 events (66.88% of the subset), while both the testing and validation sets each contain 188,967 events (16.56% each). Each subset consists of 16 columns, with the training and validation subsets capturing benign activity logs, and the testing subset containing both benign and attack activity.

Table 1

Data Dictionary for BETH Dataset

Features/Labels	Type	Description
USERID	Integer	Indicates the user who initiated the process
PROCESSNAME	String	Name of the command executed
EVENTID	Integer	Identifier for the system event
EVENTNAME	String	Name of the system event recorded in the log
SUS	Integer (0 or 1)	Binary label indicating suspicious event (1 = suspicious or 0 = not suspicious)
EVIL	Integer (0 or 1)	Binary label indicating malicious event (0 = benign or 1 = malicious)

Note. This table lists the key columns of the BETH dataset, including their names, types, and descriptions.

The UNSW-NB15 dataset comprises both real normal and synthetic malicious network traffic. It contains 2,540,044 records and was partitioned into a training set with 175,341 records and a testing set with 82,332 records. The dataset consists of 49 features, including basic, packet-based, flow-based, and connection, which are a mix of categorical and numerical types. Also, it includes two labels: a binary attack label and an attack category label with nine categories: Analysis, Fuzzers, DoS, Backdoor, Exploits, Generic, Worms, Reconnaissance, and Shellcode.

Table 2

Data Dictionary for UNSW-NB15 Dataset

Features/Labels	Type	Description
dload	Float	Destination bits per second
ackdat	Float	Time interval between SYN-ACK and ACK packets in a TCP connection
dbytes	Integer	Destination to source bytes
sbytes	Integer	Source to destination bytes
proto	String/Nominal	Network protocol used in the connection
attack_cat	String/Nominal	Name of attack category (multiclass label)
label	Integer	Binary attack label (0 = normal, 1 = attack)

Note. This table lists the key columns of the UNSW-NB15 dataset, including their names, types, and descriptions.

A key distinction between the two datasets is that the BETH dataset captures real-world network- and system-level activity collected from operational systems. In contrast, the UNSW-NB15 dataset contains a mix of real and synthetic network traffic generated in a controlled environment. This difference could affect model generalization and performance. However, the combination of the two datasets is beneficial for this project, as it allows for a comprehensive analysis of real-world and benchmark intrusion data, which can improve model robustness and generalizability.

Data Quality

Both datasets exhibit data quality issues that were addressed during preprocessing, EDA, or modeling. Missing values were not observed, however, null values in the BETH dataset were dropped. Duplicate rows and values were identified and either dropped or retained when they represented repeated system events. The datasets contained redundant features, which were addressed through feature selection and dimensionality reduction. Both datasets exhibit

significant class imbalance: the BETH dataset contains malicious events that account for only a small percentage of the data, whereas in the UNSW-NB15 dataset, normal network traffic accounts for 87% of the data, and certain attack categories, such as Worms, are highly underrepresented. This imbalance was addressed using sampling techniques, tuned class weights, and metrics such as F1 score. Also, this dataset contains synthetic data, which may not represent realistic network behavior and could limit further analysis. Some categorical features required encoding for modeling.

Ethical Considerations

The datasets used in this project contain cybersecurity-related network and system activity logs. They represent real network traffic behaviors and system activity, and thus, the data must be handled responsibly and used appropriately. The goal of the project is to improve intrusion detection and strengthen cybersecurity defenses, rather than exploit vulnerabilities. Hence, proper data handling and ethical research approaches help to ensure that this analysis is beneficial to cybersecurity research and network protection.

Data Cleaning, Preprocessing, Exploratory Data Analysis (EDA)

Missing Values, Outliers, and Inconsistencies

The BETH and UNSW-NB15 datasets do not contain any missing values. However, null values were identified in the BETH dataset and dropped to eliminate noise and reduce bias. Duplicate values in some columns were also found in BETH's training, testing, and validation sets. For example, system-level identifier columns such as `userId`, `processId`, and `eventId` had more than 762,000 duplicate values in the training set and more than 188,600 in the test set. This is not indicative of any errors, but rather a typical characteristic that occurs in the kernel logs, where repeated system events are standard. During preprocessing, some features were dropped

due to redundancy, high cardinality, data leakage, and class imbalance. These included `parentprocessId`, `threadId`, `processId`, `args`, `timestamp`, and `evil`. These removals ensured that the models learned from meaningful features rather than variables that could introduce leakage, improving overall data integrity. In the UNSW-NB15 dataset, duplicate rows were found in both the training and testing sets, totaling 78519 and 32361, respectively. These were dropped to ensure the data is clean and to avoid potential bias in further analysis and model training. Additionally, duplicate values were found in both datasets, which is normal since network traffic data typically contains repeated values. Outliers were identified in both datasets, such as extreme values in numeric columns, such as `eventId`, `processId`, and `threadId` in BETH, and in `sbytes`, `dbytes`, and `dur` in UNSW-NB15. These values represent unusually high or rare events in network traffic. These were not removed because they reflect authentic behavior that could be malicious or anomalous, which is vital in building models for threat detection. Also, high correlations between certain features and the target label were observed in both datasets, which were addressed by either removing these features or applying PCA and regularization to mitigate overfitting and data leakage.

Feature Engineering

Feature engineering was crucial in this project, as it improved predictive model performance and generalization. `StandardScaler`, a scaling technique, was applied to numeric features in both datasets to normalize feature distributions and reduce overfitting. Multiple encoding techniques were used to handle categorical variables appropriately. `OrdinalEncoder` was applied to ordered categorical features such as `processName` in BETH, while one-hot encoding was used for nominal variables. For high-cardinality features, target or frequency encoding was used to reduce dimensionality and capture meaningful patterns. These techniques

helped to reduce bias in dominant features and balance feature influence across the datasets, improving model performance in accurately detecting malicious or suspicious activity.

Dimensionality Reduction

Dimensionality reduction was applied to both datasets to remove feature redundancy, reduce noise, and improve model performance. Principal Component Analysis (PCA), a dimensionality reduction technique, was applied to the numeric features to capture most of the variance while reducing the total number of features. For the BETH dataset, PCA reduced correlated features to a smaller set of components without losing critical information, making the dataset easier for the model to learn from. For the UNSW-NB15 dataset, PCA helped determine whether dimensionality reduction would be beneficial in preserving classification performance, without losing predictive information.

Visualizations

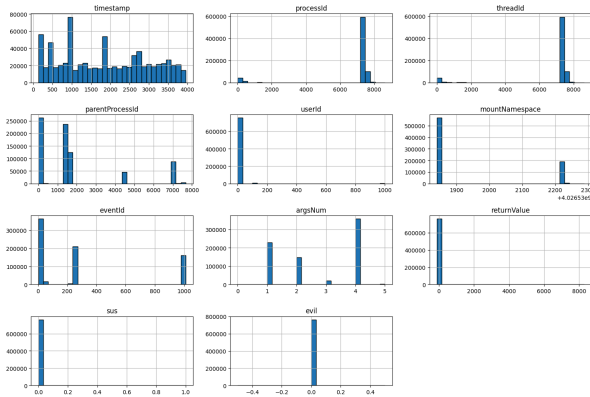
Multiple visualizations were used in the EDA process to capture feature patterns, distributions, and relationships. Some of these include histograms, scatterplots, and correlation matrices. In both datasets, histograms showed distributions of numeric features, such as eventId, processId, and threadId in BETH, and sbytes and dbytes in UNSW-NB15. These histograms revealed that most features in the UNSW-NB15 dataset were right-skewed, with occasional high values representing anomalies or attacks. Scatterplots were used to capture and understand the relationships between the features and targets, revealing strong correlations between userId and suspicious events in the BETH dataset. Correlation matrices were used for feature selection to help identify redundant features.

Figure 1

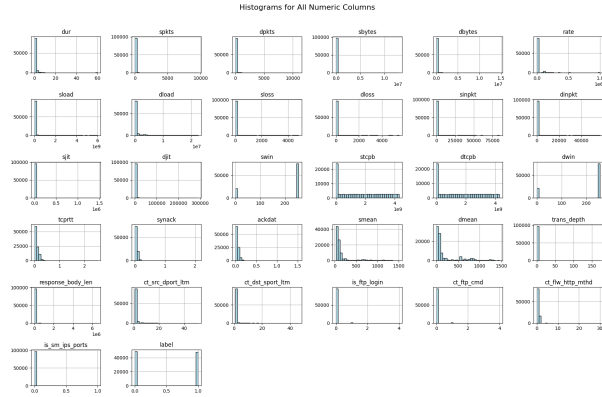
*Numeric Feature Distributions Histogram
(BETH Dataset)*

Figure 2

*Numeric Feature Distributions Histogram
(UNSW-NB15)*



Note. The Histogram depicts the distribution of numeric features in the BETH dataset. Most features are right-skewed with occasional extreme values, indicating possible suspicious behavior.



Note. The Histogram depicts the distribution of numeric features in the UNSW-NB15 dataset. Most features are right-skewed with a few extreme values, indicating possible abnormal or malicious traffic.

Key Findings

In the BETH dataset, severe class imbalance and data leakage were identified during EDA and addressed before modeling through preprocessing and feature selection. The binary labels, ‘evil’ and ‘sus’, also exhibited class imbalance. This was mitigated by using the ‘sus’ label as the target label for training, as it was less imbalanced. Key features for predicting suspicious activity were identified, such as processName, eventName, and hostName. Numeric features were mostly right-skewed, as shown in Figure 1, and extreme values were retained to capture potential anomalies associated with suspicious behavior. In the UNSW-NB15 dataset, class imbalance and data leakage were also found, which were addressed during EDA. Key features for predicting attack events included ackdat, dload, and dbytes. Numeric features were mostly right-skewed, as shown in Figure 2, and extreme values were kept as they could reflect potential anomalies or attack events.

Modeling/Analysis

Model Choices

For this project, diverse machine learning models were implemented to capture both linear and nonlinear patterns in network and system activity. Also, they were evaluated to gain insights into how differing models perform on intrusion detection tasks. Both supervised and unsupervised methods were used to provide a comprehensive comparison across classification and clustering techniques. Supervised methods, such as logistic regression, K-Nearest Neighbors (KNN), and support vector machines (SVMs), were incorporated as baseline classifiers as they are easy to interpret and are effective at separating classes using linear or boundary-based decision rules. Logistic regression provides a simple linear baseline for distinguishing between normal and suspicious or attack activity. KNN determines patterns based on similarity between data points, which is useful for detecting similar attack behaviors. SVM is effective for high-dimensional data and can separate complex class boundaries where normal and suspicious traffic overlap. Ensemble tree-based methods such as Random Forest, HistGradientBoostingClassifier, and XGBClassifier were applied because they are highly effective at identifying complex nonlinear patterns and feature relationships. They are robust to overfitting and perform well under class imbalance, making them stable and reliable models for threat detection. Unsupervised techniques such as K-Means, DBSCAN, HDBSCAN, and Hierarchical Agglomerative Clustering (HAC) were used to group similar behaviors and uncover hidden anomalous patterns and clusters in the data without relying on the target labels. K-Means was implemented to identify general behavioral groupings in network and system activity. Density-based methods, such as DBSCAN and HDBSCAN, were included as they help detect outliers or abnormal activity through clustering. HAC was applied to explore hierarchical

relationships within network and system behaviors, capturing a hierarchical representation of the data structure. Incorporating these models into the project allowed for a comprehensive analysis and comparison of different machine learning algorithms and helped identify which of these algorithms were most effective in answering the project's objective and guiding research questions. Table 3 below summarizes the model choices to facilitate comparison of their roles in the analysis.

Table 3

Model Summary

Model	Type	Task	Key Characteristics
Logistic Regression	Supervised	Classification	Highly interpretable, effective for establishing linear separability and reducing overfitting in high-dimensional data
K-Nearest Neighbors (KNN)	Supervised	Classification	Captures local patterns in network traffic, useful for identifying similar attack behaviors
Support Vector Machine (SVM)	Supervised	Classification	Effective in high-dimensional data, it performs well in separating complex classes
Random Forest, HistGradientBoosting, XGBoost	Supervised	Classification	Captures nonlinear relationships and features interactions well, robust to class imbalance and overfitting, and performs well in identifying anomalies

K-Means	Unsupervised	Clustering	Groups similar network behaviors without labeled data and useful for identifying general traffic patterns
DBSCAN, HDBSCAN	Unsupervised	Clustering	Identifies noise and anomalies, irregular attack patterns, and clusters for suspicious activity
Hierarchical Agglomerative Clustering (HAC)	Unsupervised	Clustering	Provides a hierarchical structure of network behavior and is useful for understanding layered threat patterns

Note. This table provides an overview of the machine learning models used in this project for analysis. It summarizes the types of learning, model tasks, and key characteristics that describe their roles in intrusion detection and network activity analysis.

Modeling Training Process

The training process for each model followed a similar structured machine learning workflow. After preprocessing and feature engineering, models were trained using scaled numeric features, one-hot-encoded categorical variables, and ordinal-encoded features. This ensured consistent inputs across all algorithms and improved overall model performance. For supervised models, the target variable used for the BETH dataset was the ‘sus’ label, as it is more balanced and has a higher frequency in the training and validation sets compared to the ‘evil’ binary label. This improved model stability and reduced bias towards dominant classes. For the UNSW-NB15 dataset, both the binary classification (normal vs attack) and the multiclass labels (attack_cat) were used for supervised algorithms to distinguish between normal and attack events and to identify specific attack types. Models were initially trained without cross-validation or

hyperparameter tuning to achieve a baseline performance and assess how well they learned patterns in each dataset. This step provided insight into each model's ability to capture underlying patterns before hyperparameter tuning. Also, this helped identify issues with data quality or the need for further preprocessing. Following the assessment of the models' baseline performance, cross-validation and hyperparameter tuning were applied to optimize performance. Specifically for the BETH dataset, the training and validation sets were concatenated after tuning to train the models, improving learning before the final assessment.

Data Splitting and Cross-Validation

The BETH dataset was initially partitioned as 70% training, 15% validation, and 15% testing. This split ensured that the models could learn from most of the data in the training set. The validation set was used for hyperparameter tuning, and the test set was held out to obtain an unbiased assessment of the model's generalization to unseen data. After initial hyperparameter tuning, the training and validation sets were concatenated, yielding an 80% training, 20% testing split. A 5-fold stratified cross-validation was applied to the training set to handle class imbalance and ensure that the proportion of the 'sus' label was maintained across folds. This improved the model's learning, while maintaining a sufficiently sized testing set for final evaluation. For the UNSW-NB15 dataset, an initial data split of 80% training and 20% testing was applied. However, to handle class imbalance, 5-fold stratified splitting was used during cross-validation to ensure that proportions of the binary target and multi-class labels are consistently maintained and avoid underrepresentation of minority classes.

Evaluation Metrics

For both datasets, model performance in supervised methods was evaluated using metrics that account for class imbalance. The metrics used were precision, accuracy, recall, F1 score,

macro F1, and ROC AUC. For supervised learning techniques, an emphasis was placed on macro F1 and recall, as these metrics ensure that the minority classes, such as attack instances, are not overlooked. Recall reduces false negatives, which is critical in intrusion detection tasks, as missed suspicious or attack events could result in data breaches. Macro F1 ensures a balanced performance across the majority and minority classes. ROC AUC was also used to assess how accurately models distinguished between classes at various decision thresholds, providing a measure of class separability. This is especially crucial when comparing models whose performance varies across different decision boundaries. For unsupervised algorithms, silhouette scores, cluster counts, and WCSS were used to assess cluster quality. These metrics evaluate cohesion, separation, and the consistency of identified behavioral clusters, allowing for the comparison of clustering effectiveness in detecting meaningful network patterns. Overall, the metrics used in this project enabled consistent comparison of model performance and provided insights into their ability to effectively distinguish between suspicious and normal traffic, generalize well to unseen data, and capture key patterns.

Hyperparameter Tuning

Hyperparameter tuning was applied to both supervised and unsupervised models to improve predictive performance and reduce overfitting. For logistic regression and SVMs, the regularization strength (parameter C) was tuned using `RandomizedSearchCV` to control model complexity, thereby reducing overfitting, improving generalization to unseen data, and ensuring stable convergence. For ensemble methods, including `Random Forest`, `HistGradientBoostingClassifier`, and `XGBClassifier`, parameters such as `n_estimators`, `max_depth`, and `min_samples_split` were tuned using `RandomizedSearchCV`. These parameters control model complexity and learning capacity, which allows models to effectively capture

nonlinear relationships while mitigating overfitting. In K-Means, the number of clusters (parameter k) was tuned through the Elbow and Silhouette methods to identify the optimal number of clusters that best capture patterns in the network traffic. This showed optimal k -values around 4 for the BETH dataset and 2 for UNSW-NB15. In DBSCAN and HDBSCAN, parameters such as `eps`, `min_cluster_size`, and `min_samples` were tuned using silhouette analysis to identify dense regions and separate unusual clusters, thereby improving anomaly detection. This showed that HDBSCAN was effective in producing interpretable clusters and distinguishing meaningful patterns in network traffic. For HAC, the Ward linkage parameter was used to reduce variance within clusters, yielding interpretable and compact hierarchical structures of network activity. Overall, hyperparameter tuning improved model stability and slightly increased performance metrics, such as recall and macro F1 in the ensemble models. Specifically, `n_estimators` and `max_depth` had the greatest impact, improving generalization to unseen data while controlling model complexity. Table 4, below, provides an overview of the hyperparameters and tuning methods used for each model.

Table 4

Hyperparameter Tuning Settings

Model	Key Parameters	Tuning Method	Purpose
Logistic Regression, SVM	C (regularization strength), kernel parameters (SVM)	RandomizedSearchCV	Controls model complexity and improves generalization in high-dimensional, imbalanced data
Random Forest, XGBoost	<code>n_estimators</code> , <code>max_depth</code> , <code>learning_rate</code> , <code>min_samples_split</code> , <code>subsample</code> ,	RandomizedSearchCV and manual tuning	Improves generalization, robustness to overfitting, and class imbalance

	colsample_bytree		
HistGradientBoosting	max_iter, learning_rate, max_leaf_nodes, min_samples_leaf, l2_regularization, class_weight	Manual tuning and class weighting	Enhances stability and handles class imbalance while modeling complex patterns
K-Means	n_clusters (k)	Elbow Method and Silhouette Score	Identifies optimal grouping of network behavioral patterns
DBSCAN, HDBSCAN	eps, min_samples, min_cluster_size	Empirical tuning and silhouette analysis	Detects dense clusters, anomalies, and noise points
HAC	linkage (Ward)	Model Selection	Provides hierarchical structures of cluster relationships

Note. This table summarizes the different parameters, tuning methods, and optimization goals for each machine learning model used in the analysis.

Results

Model Comparison and Performance Metrics

Model performance in this project is assessed based on how well each model satisfies the success criteria, particularly in identifying suspicious and attack behavior for the BETH and UNSW-NB15 datasets. Table 5 below presents the comparative performance of the supervised models across both datasets.

Table 5

Model Performance Results

Dataset	Model	Precision	Test Accuracy	Recall	F1 Score	Macro F1	ROC AUC
BETH	Random Forest	0.990	0.945	0.950	0.970	0.970	0.987

BETH	HGB (HistGradientBoosting)	0.970	0.969	0.970	0.969	0.945	0.987
BETH	XGBClassifier	0.970	0.969	0.970	0.969	0.910	0.985
UNSW-NB15	Random Forest	0.840	0.850	0.980	0.910	0.840	0.840
UNSW-NB15	HGB (HistGradientBoosting)	0.920	0.923	0.920	0.924	0.924	0.890
UNSW-NB15	XGBClassifier	0.810	0.812	0.810	0.812	0.812	0.812

Note. The table shows performance metrics for the models that performed best on the BETH and UNSW-NB15 datasets. Metrics include precision, test accuracy, recall, F1 score, Macro F1, and ROC AUC.

Random Forest performed best on the BETH dataset, achieving the highest precision, Macro F1, and ROC AUC scores, indicating strong overall classification performance.

HistGradientBoosting outperformed the others on the UNSW-NB15 dataset, achieving the highest scores across most of the metrics, demonstrating better generalization to more complex and less predictable attack patterns.

Feature Importances

Feature importance analysis was used to determine the most influential variables contributing to model predictions and to improve the interpretability of intrusion detection decisions. In the BETH dataset, initial results showed that `userId` had the highest feature importance score. However, this feature was excluded from model retraining and final analysis because it conflates measurement by potentially serving as an identifier rather than a true behavioral signal. Retaining such a feature could have led to data leakage or inaccurate

conclusions. After excluding `userId`, the most influential features included `processName`, `hostName`, and `returnValue`. The `processName` feature captures the type of process executed, which helps distinguish between normal and suspicious activity. The `returnValue` feature represents the outcome of system calls, providing insight into whether executions succeeded or failed and indicating abnormal behavior. The `hostName` feature contributed to model performance by providing system-level context, but was interpreted cautiously as it may not fully reflect actual behavior. For UNSW-NB15, some of the most important features included `ackdat`, `dbytes`, and `dload`. `Ackdat` refers to the time interval between SYN-ACK and ACK packets in a TCP connection, which helps to identify abnormal connection behavior. `Dbytes` is the destination to source bytes, which indicates unusual data transfer patterns that are often linked to suspicious activity. `Dload` refers to destination bits per second, which can highlight suspicious traffic patterns. These features were consistent across the best-performing models mentioned above. This indicates that these features are reliable predictors of suspicious or attack behavior, which can help models accurately detect potential threats when observing network traffic.

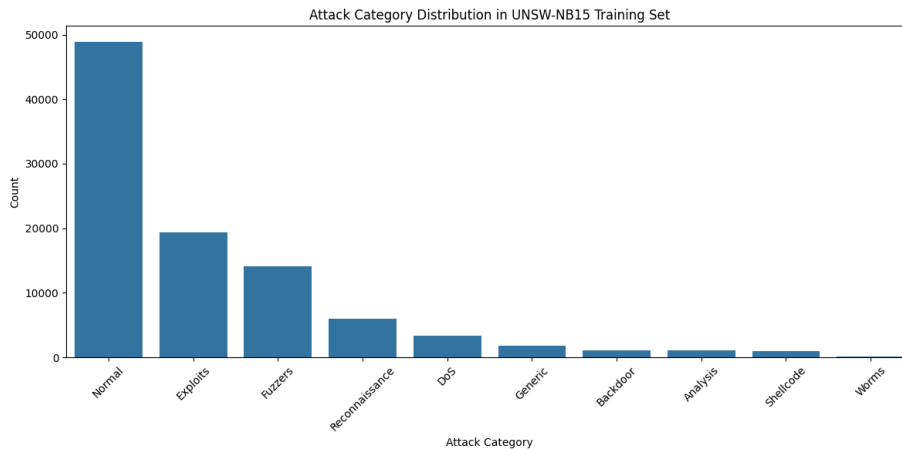
Attack Category Distribution and Analysis for UNSW-NB15 Dataset

The UNSW-NB15 dataset exhibited a significant imbalance in the distribution of attack categories, which was mitigated but may still have affected model learning and performance. The most frequent attack types in the dataset are Exploits (19360 counts) and Fuzzers (14082), followed by Reconnaissance (6000) and DoS (3369), indicating that network intrusions are concentrated in a small set of recurring attack types rather than evenly distributed across all categories. In contrast, categories such as Shellcode and Worms occur rarely, suggesting that these attacks are less common but potentially more severe. The results show that attack behavior is not uniformly distributed across protocol types. Specific attack categories are strongly

associated with certain protocols, particularly within a small subset of protocol values, such as INT and FIN. This indicates that attack activity follows structured, repeated communication patterns rather than random behavior, supporting the ability of machine learning models to distinguish attack types based on protocol information. From a modeling perspective, this distribution helps explain the observed performance of the ensemble tree-based models used in the project. These models are well-suited to capture complex, nonlinear interactions between features and are more robust to class imbalance. This allows them to effectively learn both majority and minority attack patterns. As a result, they achieved a stronger performance in distinguishing between normal and attack events.

Figure 3

Distribution of Attack Types in UNSW-NB15 Dataset



Note. The chart provides the distribution of attack types for the UNSW-NB15 dataset. It shows severe class imbalance with ‘Normal’ traffic dominating, followed by ‘Exploits’ and ‘Fuzzers’, while categories such as ‘Shellcode’ and ‘Worms’ are underrepresented.

Performance Visualizations

To complement the quantitative results from Table 5, the precision-recall curves below were used to further evaluate ensemble tree-based models under class imbalance conditions. The

curves reinforce the robustness of these models in identifying complex patterns while maintaining stable precision-recall performance across imbalanced datasets.

Figure 4

Precision-Recall for BETH Dataset

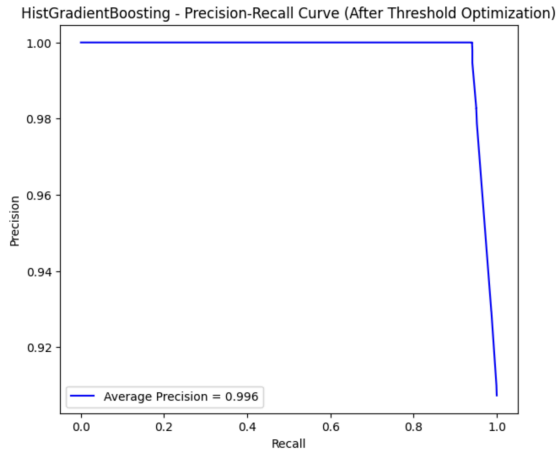
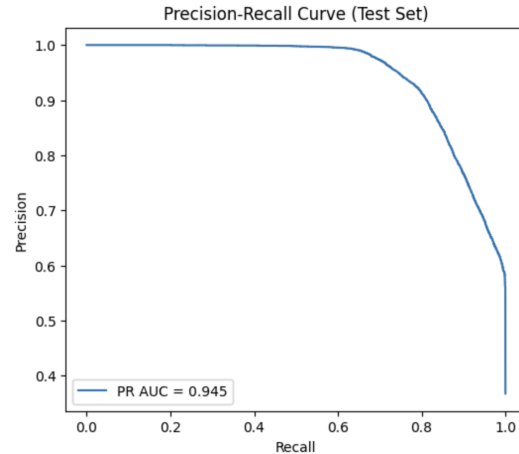


Figure 5

Precision-Recall for UNSW-NB15 Dataset



Note. The precision-recall curve shows the relationship between precision and recall for the BETH dataset in detecting suspicious events. The HGB model achieves high precision across most recall values, indicating strong performance in detecting suspicious events.

Note. The precision-recall curve shows the relationship between precision and recall for the UNSW-NB15 dataset in detecting attack events. The HGB model exhibits relatively high precision as recall increases, indicating that it performs strongly at detecting attack activity from normal.

Comparative Analysis of Models and Limitations

After evaluating the supervised and unsupervised models used in this project, it is evident that ensemble tree-based models, such as Random Forest, HistGradientBoosting, and XGBClassifier, outperformed other models, such as logistic regression and KNN. On the BETH

dataset, Random Forest achieved a Macro F1 score of 0.970 and a ROC AUC score of 0.987. This indicates a strong balance between detecting suspicious or attack behavior, while minimizing false negatives. This suggests the model is particularly effective under conditions where class imbalance is present, provided the underlying relationships remain consistent. HistGradientBoosting and XGBClassifier showed Macro F1 scores of 0.945 and 0.910, respectively, indicating they were slightly less effective at distinguishing between classes. For UNSW-NB15, HistGradientBoosting outperformed the other models, with a Macro F1 score of 0.924 and an ROC AUC of 0.890, whereas XGBClassifier had a Macro F1 score of 0.812 and an ROC AUC of 0.812. This indicates that HistGradientBoosting generalized well to the unseen data and was able to distinguish between normal and suspicious events equally. These results indicate how data structure and characteristics can influence model performance, and how ensemble tree-based models can effectively handle class imbalance and overfitting for detecting threats in network traffic, as they can separate minority patterns at deeper splits. While these models performed well, a possible limitation is that they may struggle with data containing new or rare attack patterns that were not seen during training. Another limitation is that these models assume that new network traffic will behave similarly to previous data, which could lead to a loss in detection accuracy. Also, ensemble tree-based models are more difficult to interpret compared to simpler models.

Business Implications

These results indicate that implementing machine learning models, specifically ensemble tree-based approaches, can help organizations uncover complex patterns in cybersecurity data, enabling more effective detection and mitigation of potential threats. Models such as Random Forest and HistGradientBoosting demonstrated strong performance in identifying suspicious and

attack activity, even under severe class imbalance conditions. This enables more reliable detection of anomalous or rare threats, supports earlier threat detection, and improves the prioritization of high-risk behaviors. As a result, organizations can allocate resources more effectively to high-risk systems, strengthen system defenses, and reduce the overall risk and damage of cyber threats.

Recommendations

Based on the analysis and key findings of this project, it is recommended that clients implement ensemble tree-based models such as Random Forest and HistGradientBoosting in their network monitoring systems. These models are effective in identifying complex and unusual patterns within network traffic. Integrating such models can help cybersecurity professionals detect characteristics or behaviors deemed suspicious efficiently, allowing them to react more proactively. From a business perspective, organizations should incorporate these algorithms into their current security infrastructures to continuously monitor network traffic for potential threats. Using automated detection systems with these models allows security professionals to prioritize high-risk threats and focus on investigating activity that may exhibit suspicious behavior. This approach helps organizations respond in a timely manner, lowers the risk of potential network intrusions, and strengthens overall security defenses. To ensure that the implementation of these models is most effective, clients should develop a continuous monitoring and retraining workflow to keep models updated with new network patterns and emerging or novel threats. It is also recommended to maintain clean, high-quality, and balanced datasets by regularly auditing and cleaning incoming logs. This will improve model performance and ensure that suspicious behavior and threats are being accurately detected. Also, incorporating a feedback loop, in which professionals can review and correct model predictions, can help

improve model training and accuracy. For further analysis, organizations should collect new data, such as features that capture endpoint activity, user behavior metrics, or detailed network traffic statistics, which can enhance the models' ability to detect emerging and complex threats.

References

- Ahmed, F. (2025). *Milestone 3: Exploratory Data Analysis for Network Threat Detection Through Anomalies and Attack Classification [Unpublished manuscript]*. Boston University
- Agur, A., & Venugopal, U. (2023). *Cyber Security Attacks*. Kaggle. Retrieved June 29, 2025, from <https://www.kaggle.com/datasets/teaminciribo/cyber-security-attacks/data>
- Highnam, K., Hanif, Z., & Arulkumaran, K. (2021). *BETH Dataset*. Kaggle. Retrieved June 29, 2025, from <https://www.kaggle.com/datasets/katehighnam/beth-dataset>
- Moustafa, N., & Slay, J. (2015, November). *UNSW-NB15*. Kaggle. Retrieved June 29, 2025, from <https://www.kaggle.com/datasets/dhoogla/unswnb15>

AI Appendix

AI tools like ChatGPT and Grammarly were used to correct grammar and check if the papers aligned with the rubric. Various sections were reviewed using these tools to help organize the paper and improve clarity. Here is a link to my conversation with ChatGPT:

<https://chatgpt.com/share/69e59bfc-ef18-83e8-89e8-9e472c7cc7e1>.

**Non-Technical Report: A Data-Driven Approach to Enhancing Cybersecurity Through
Threat Detection**

Farah Ahmed

Data Science MS Online, Boston University

DX 799 O2: Data Science Capstone

Joshua Von Korff

April 19, 2026

Executive Summary

As organizations heavily rely on digital systems, they are responsible for protecting sensitive information, maintaining secure operations, and proactively strengthening security measures to prevent cybersecurity threats. However, as cyberattacks have become more prevalent and complex, organizations face difficulty detecting and mitigating them. Traditional security measures are often limited in their ability to identify evolving threats due to their reliance on manual processes and predefined rules. This project focuses on how data-driven techniques, including machine learning, can help detect and better understand the underlying patterns of malicious activity, thereby improving cybersecurity. The primary objective of this project is to identify the most common types of cyberattacks and to develop an effective approach to distinguishing suspicious activity from normal. To achieve this, datasets representing real-world system activity and network traffic were analyzed. These datasets contain information on how systems behave under both normal conditions and potential cyberattacks, providing crucial insights into signals that are strongly associated with suspicious or malicious activity. Key findings from this analysis show that data-driven methods and machine learning are highly effective in identifying potential threats and reducing inaccurate threat detections. Specific characteristics in system and network activity were found to be valuable indicators of malicious or suspicious behavior. Also, the analysis revealed which types of cyberattacks were most frequent. Based on these findings, the strongest recommendations for organizations are to implement automated, data-driven monitoring and threat detection systems to continuously detect potential threats, regularly maintain and improve these systems to better adapt to emerging threats, and prioritize high-risk system behaviors to strengthen security defenses further. These recommendations can help organizations avoid costly risks, reduce system disruptions or security

vulnerabilities, and overall, enhance security defenses. This enables businesses to shift from a reactive to a more proactive and resilient approach to managing cybersecurity risks.

Introduction and Problem Definitions

Cybersecurity plays a crucial role in today's digital age, as organizations and individuals depend on digital systems to store, process, and protect sensitive information. However, with the increasing reliance on these systems, cyber threats have become more frequent and sophisticated. These attacks include malicious or suspicious activity that targets and exploits system vulnerabilities, such as unauthorized access, data theft, and abnormal network behavior. This often compromises data integrity, weakens essential system functions, and threatens people's privacy. Due to these challenges, organizations require effective measures to detect and understand threat patterns. Traditional security methods alone are insufficient to handle the complexity of rapidly evolving cyberattacks. Machine learning can help address these challenges by analyzing system and network data to identify patterns and unusual behavior that are indicative of potential threats, distinguish between normal and abnormal activity, and adapt to emerging cyberattacks.

The main goal of this project is to use data-driven approaches, such as machine learning, to develop methods that can improve the detection and understanding of cyber threats. Specifically, this project aims to identify the most common types of cyberattacks and develop a reliable method for distinguishing suspicious behavior from normal system activity. To further support this objective, the analysis also considers which machine learning techniques help improve the detection of suspicious network behavior and which patterns are the strongest indicators of potential cyber threats. To achieve this, machine learning models, which are algorithms that learn patterns from observing data and then make predictions on new data, were

built and assessed using datasets that capture real-world network and system activity. The success of the analysis is evaluated by how well the machine learning models accurately and consistently identify suspicious or malicious activity while avoiding the incorrect flagging of normal behavior as a threat. Model performances were assessed using standard evaluation metrics. This success criterion indicates that the model can effectively detect cyberattacks in real-world scenarios and provide meaningful insights that organizations can use to respond to potential threats and proactively strengthen their cybersecurity defenses.

Data Overview

This project uses two datasets, BETH and UNSW-NB15, which contain real-world and synthetic cybersecurity activity. They provide information on how computer systems and networks function under normal conditions and when experiencing malicious or suspicious activity. These datasets were collected from Kaggle, a data science platform that hosts datasets on various topics. The BETH dataset contains millions of records of cyberattacks and system activities such as user actions and system processes. It also uses labels to categorize the records as either normal, suspicious, or evil (malicious), helping to identify patterns of potential threats. The UNSW-NB15 dataset contains network traffic, which is the flow of data moving across computers and devices. This dataset provides information about communication between systems, such as data transfers and connections, uses labels to categorize network activity as either normal or an attack, and identifies the type of attack. Both datasets include key variables or features describing how systems operate and communicate during normal and suspicious activity. Together, these two datasets provide complementary perspectives on system and network behavior that can help improve the understanding and detection of cyber threats.

For this project, the datasets were reviewed to address any data quality and consistency issues. Duplicate entries were found and were either removed or retained, depending on whether they represented meaningful data. Also, the datasets contained unnecessary or repetitive information that was removed to improve data quality. A significant issue in both datasets was a severe imbalance between normal and threat-related activity, with normal behavior occurring more often than attack behavior in the recorded data. This can make it challenging for machine learning models to detect less common yet crucial cyber threats accurately. This imbalance was addressed to ensure reliable analysis. Ethical considerations were also taken into account by handling the data responsibly and appropriately, given that it contains real network data and system activity. Ethically sound research approaches guided this project, ensuring that the analysis supports the development of stronger cybersecurity defenses.

Data Cleaning, Preprocessing, and Exploratory Data Analysis (EDA)

Before developing the machine learning models, the datasets were prepared to ensure that they were consistent, accurate, and suitable for further analysis. This process is crucial because most cybersecurity datasets contain errors, inconsistencies, and redundant information that can lead to unreliable results and analyses. The datasets were reviewed for missing values, outliers, and any inconsistencies. No missing values were found, however, duplicate values were identified in both datasets, which were retained as they represented typical network or system behavior. Outliers were also observed, such as extreme or unusual values, which represent rare, yet important system or network events. These were not removed because they represent real suspicious or malicious behavior, which is vital for building and training models to detect cyber threats accurately. Redundant or unnecessary information, such as features in the datasets, was removed to reduce data complexity and help avoid potential bias in the analysis. These steps

helped to improve the quality and reliability of the datasets. To better prepare the datasets, some information was transformed into a format that was easier for the models to interpret. This involves converting text-based information, such as system instruction names, into a numeric format that allows models to process the data effectively. Also, numerical information was adjusted so that no variable or feature would dominate the results. The number of features was also reduced to simplify the datasets while preserving the most significant information, making it easier for models to learn. Exploratory data analysis, including visualization methods such as graphs and charts, was then applied to better understand the datasets. These visualizations helped to capture the distribution of the data, the relationships between features, and any important or unusual patterns.

These steps revealed key findings. Both datasets exhibited a severe imbalance between normal and threat-like activity, with normal behavior occurring more frequently. This is a common issue in cybersecurity data, making it difficult to detect cyber threats effectively. Despite this, key indicators and patterns were identified, such as unusual network behaviors, which can help predict suspicious or attack-related activity. Overall, these steps ensured that the data used for this project was well prepared to support the development of reliable models for threat detection.

Modeling/Analysis

A variety of models were used for this project to address the goal of detecting and better understanding cyber threats. They were evaluated to determine which performed strongly at identifying patterns of normal and suspicious or malicious network behavior. Model techniques, including those designed to predict outcomes from learning patterns using labeled data and those that find patterns and group similar behaviors without relying on labeled data, were applied. This

allowed for a comprehensive analysis and comparison of different machine learning models, which helped to identify the most effective and suitable models for addressing the project's goals. The models were trained using the cleaned and prepared datasets. The datasets were divided into separate portions so that the models could train on one portion, adjust and improve on another, and then be tested on the final portion. The models were also evaluated using a method known as cross-validation, which tests the models on different subsets of the data to yield more consistent and reliable results. This ensures that models can accurately predict outcomes on new data. To further improve model performance, various model settings were adjusted and evaluated to determine which set of configurations produced the most accurate and reliable results. Models were first assessed with initial settings and then refined to improve performance in detecting threats while minimizing inaccuracies. Evaluation metrics were used to quantify model performance by how well each model accurately predicts suspicious or malicious behavior from normal behavior, including how reliably they perform on new data. This helps to compare models, guide improvements in model development, and address any imbalance between normal and attack activity. These steps were vital for determining which machine learning techniques were most effective for improving cyberattack detection and identifying key patterns in the data.

Results

After models were refined and tested, their results were compared to determine which models were the most accurate and reliable in detecting cyber threats. This assessment focused on how effectively these models can distinguish attack behavior from normal behavior, how consistently they can perform on new data, and their ability to identify meaningful patterns. Various measures were used to compare model performances, including accuracy (overall

correctness), precision (the ability to identify threats correctly), recall (how well the model can capture all actual threats), F1 score (a balance between precision and recall), macro F1 score (similar to F1 score, but treats normal and threat-like behavior equally even when imbalance is present), and ROC AUC (how well the model can separate between normal and threat-like activity). These measures help provide a comprehensive analysis of how well each model performs with real-world detection scenarios.

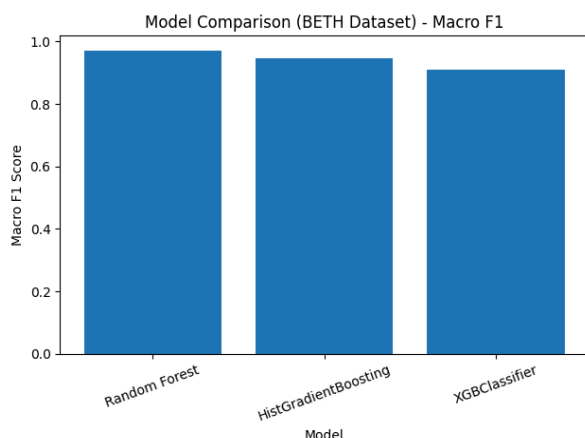
Performance results showed that ensemble tree-based models performed the strongest. These models use a flowchart-like structure that combines multiple simple decision-making processes and then averages them. Rather than depending on a single decision path, these models incorporate many different decision paths to improve accuracy and stability. This makes them well-suited for complex data, such as cybersecurity activity in the BETH and UNSW-NB15 datasets, where patterns may not be straightforward. Furthermore, they are less sensitive to outliers and help reduce bias. For the BETH dataset, Random Forest, an ensemble tree-based model, achieved the highest performance, demonstrating a strong ability to detect suspicious activity while maintaining balanced results for both normal and suspicious behavior. This suggests that the model is highly effective at identifying threats without many errors and performs well on new data. For the UNSW-NB15 dataset, the HistGradientBoosting, another ensemble tree-based model, performed the best. It showed a strong ability to distinguish between normal and attack behavior, while remaining consistent when tested on new data. These results indicate that more advanced models are necessary to handle cybersecurity tasks, as they can capture complex patterns and relationships in data. On the other hand, simpler models, which rely on single decision rules based on patterns in the data, were also evaluated and were less effective at detecting subtle suspicious or attack patterns and were more likely to miss rare

activity. These results are expected because the cybersecurity data contains nonlinear relationships and overlapping patterns, which advanced models are better suited for.

Performance Visualizations

Figure 1

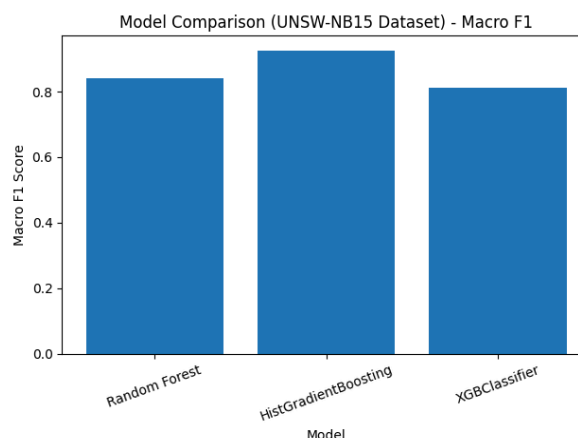
Model Comparison for BETH Dataset



Note. The bar chart compares ensemble tree-based models on the BETH dataset using the macro F1 score metric. It shows that Random Forest achieved the highest performance in detecting suspicious activity.

Figure 2

Model Comparison for UNSW-NB15 Dataset



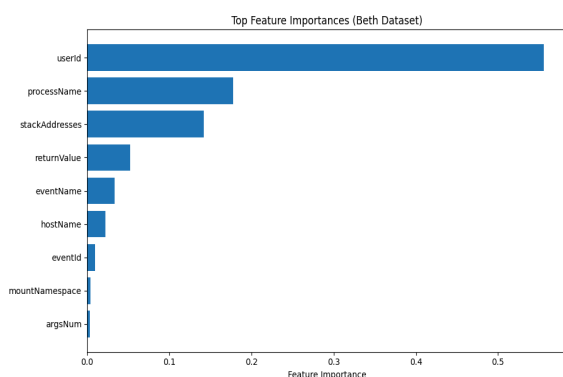
Note. The bar chart compares ensemble tree-based models on the UNSW-NB15 dataset using the macro F1 score metric. It shows that HistGradientBoosting performed the strongest in detecting attack behavior.

To better understand model behavior, feature importance, which measures which information in the data has the most influence on the model's predictions, was applied. This simply means it shows which data characteristics the models rely on most to determine whether a cybersecurity event is normal or a threat. In the BETH dataset, initial results showed that `userId` and `stackAddresses` were highly influential features. However, these were dropped as they did not reflect actual meaningful system behavior and could inflate model performance. After these were removed, the most influential features included `processName`, `eventName`, and `returnValue`. These features describe system activity, such as what processes are running, what actions are

being performed, and the outcomes of those actions. All of which can help the models make accurate predictions in detecting cyberattacks. In the UNSW-NB15 dataset, the most significant features included ackdat, dbytes, and dload. These features are related to network activity, such as the timing between network connection responses, the amount of data being transferred between systems, and the speed at which data is downloaded to a device. All of these can help models identify abnormal behavior and unusual patterns, improving their ability to detect cybersecurity threats. These features from both datasets were consistent across the best-performing models, which indicates that they are reliable indicators of suspicious or attack behavior.

Figure 3

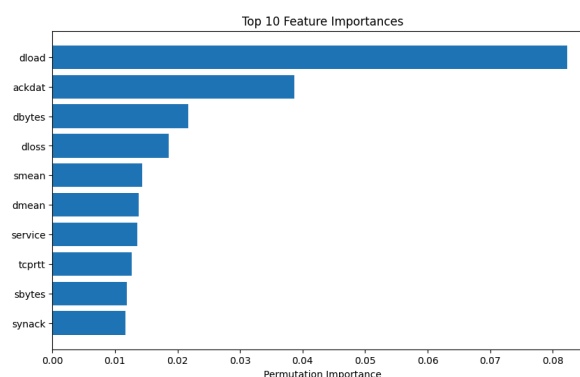
Feature Importance for BETH Dataset



Note. The bar chart shows the top 10 most important features for the BETH dataset. processName, returnValue, and eventName are the top features. userId and stackAddresses were later removed as they were not useful for prediction during model

Figure 4

Feature Importance for UNSW-NB15 Dataset



Note. The bar chart shows the top 10 most important features for the UNSW-NB15 dataset. Dload, ackdat, and dbytes are the top 3 features.

retraining.

Despite strong model performance, there are limitations that need to be considered. For example, since these models were trained on past data, they may struggle with detecting new or rare cyber threats that do not exhibit previous patterns that were observed. Another limitation is that the models assume that newer network or system behaviors will be similar to previous ones, which is untrue, as cybersecurity is rapidly evolving. Lastly, more advanced models, such as ensemble tree-based models, are often difficult to interpret as they involve many combined decision-making processes. This makes it challenging to trace specific decision paths and understand how certain features contribute to final model predictions.

From a business perspective, these results indicate that implementing machine learning models for cybersecurity tasks can be highly beneficial for organizations. These models can help detect threats more quickly and accurately than traditional cybersecurity methods. This allows organizations to proactively respond to potential attacks before they can escalate and cause further damage. Furthermore, identifying high-risk patterns, unusual or rare behavior, and key features enables organizations to better allocate resources, improve overall threat detection, and strengthen security defenses and measures.

Recommendations

Based on the project's analysis and findings, it is recommended that clients consider implementing machine learning models, specifically ensemble tree-based models such as Random Forest and HistGradientBoosting, to strengthen their security systems. Evidently, these models have shown strong performances in identifying suspicious or attack behavior within network and system activity. Integrating such models into existing cybersecurity systems can enhance organizations' ability to detect abnormal behaviors and patterns early, allowing for more

proactive responses. Automated threat detection tools that use machine learning should be used, as they can continuously monitor network and system activity in real time. These can help professionals identify high-risk or unusual activities more efficiently compared to standard cybersecurity measures, allowing them to prioritize the most critical threats. This approach is beneficial in that it can prevent potential damage and reduce the risk of data breaches or the compromise of sensitive information. Furthermore, organizations can allocate their resources and time more effectively.

To ensure these approaches are successfully implemented, clients should develop a continuous monitoring and model training workflow that regularly keeps models up to date with new network or system patterns and emerging threats. Also, it is vital that the data being used for training these models is well-prepared by ensuring that they are cleaned, balanced, and accurate. This means regularly reviewing and maintaining data, removing any extreme outliers or inconsistencies, and ensuring that both normal and threat-like activity is being equally represented. It is also recommended that human oversight be present during this process, with security professionals reviewing and correcting model predictions. This can improve model training and accuracy over time and help security systems adapt to evolving cybersecurity conditions.

For further analysis and improvements, organizations should consider collecting new data to enhance model performance, such as more detailed information capturing user behavior, network activity, and system actions. Expanding the range and quality of data is essential, as it can improve machine learning models' ability to detect complex or future cybersecurity threats. Organizations can also explore incorporating multiple models or testing other machine learning techniques to strengthen their monitoring and security workflow.

Overall, machine learning plays a vital role in cybersecurity as it can improve and strengthen the processes of detecting, understanding, and responding to cyber threats.

References

- Ahmed, F. (2025). *Milestone 3: Exploratory Data Analysis for Network Threat Detection Through Anomalies and Attack Classification [Unpublished manuscript]*. Boston University
- Ahmed, F. (2026). *Technical Report Draft: Network Threat Detection Through Anomalies and Attack Classification*. Boston University
- Agur, A., & Venugopal, U. (2023). *Cyber Security Attacks*. Kaggle. Retrieved June 29, 2025, from <https://www.kaggle.com/datasets/teamincirbo/cyber-security-attacks/data>
- Highnam, K., Hanif, Z., & Arulkumaran, K. (2021). *BETH Dataset*. Kaggle. Retrieved June 29, 2025, from <https://www.kaggle.com/datasets/katehighnam/beth-dataset>
- Moustafa, N., & Slay, J. (2015, November). *UNSW-NB15*. Kaggle. Retrieved June 29, 2025, from <https://www.kaggle.com/datasets/dhoogla/unswnb15>

AI Appendix

AI tools like ChatGPT and Grammarly were used to correct grammar and check if the papers aligned with the rubric. Various sections were reviewed using these tools to help organize the paper and improve clarity. Here is a link to my conversation with ChatGPT:

<https://chatgpt.com/share/69e598c2-a56c-83e8-8a5d-e51af28a395c>.